

# Exact optimal strategies for the WordHunt oracle-query game

## A research logbook

Started 2026-09-17

## 1 Motivation

`wordhunt.py` / `wordhunt2.py` (in the parent directory) are recreational assistance tools for a daily puzzle: 25 letters (one letter of the alphabet entirely absent) fill a  $5 \times 5$  grid; a single 5-letter dictionary word is spelled along one of 12 lines (5 rows, 5 columns, 2 diagonals); the player learns, one query at a time, whether a chosen letter is part of the answer word and, if so, at which grid cell.

Every statistic those tools compute — raw letter frequency, Zipf-weighted frequency, positional entropy, a proposed Bayesian “leading candidate” posterior — turned out, on inspection, to be a *one-step heuristic* resting on an unstated and unverified assumption about how the grid was generated. Repeated design discussion (2026-09-17) concluded that heuristics of this kind cannot be validated or compared meaningfully until at least one fully rigorous baseline exists: a strategy that is *provably optimal* for an *explicitly stated* generative model  $M$  and an *explicitly stated* objective  $L$ .

This logbook is that project. The plan (agreed 2026-09-17) is:

1. Shrink the game to a size where every board, and every possible play-through, can be exhaustively enumerated by brute force — so every claim can be machine-checked, not just argued.
2. Get a genuinely optimal solver working and validated there.
3. Scale up board size step by step, tracking exactly where and why brute-force techniques stop being tractable, and look for workarounds at each wall before conceding defeat.
4. Keep this document updated as a persistent record, so a negative result (“exact solving becomes impractical beyond size  $k$ , for reason X”) is as valid and as carefully written up as a positive one.

All code lives in `research/` on the `optimal-solver-research` git branch, kept separate from the recreational tools so nothing here can break them. All numeric results quoted below were produced by the code in this directory and are reproduced by `research/tests/`.

## 2 Model $M_1$ : the generative process

Parametrised by board size  $n$  (the real game is  $n = 5$ ):

- Alphabet size =  $n^2 + 1$  (exactly one letter of the alphabet is absent from every grid, mirroring the real game).

- Grid is  $n \times n$ ; **lines** (candidate word positions) are the  $n$  rows,  $n$  columns, and 2 diagonals:  $2n + 2$  in total.
- A fixed vocabulary  $W$  of length- $n$  strings, each with  $n$  distinct letters (not required to be English — just a fixed toy dictionary for small  $n$ ; the real `dict_game.csv` is used once  $n = 5$ ).

$M_1$  generates a grid as:

1. word  $w \sim \text{Uniform}(W)$
2. line  $p \sim \text{Uniform}(\text{the } 2n + 2 \text{ lines})$
3. excluded letter  $e \sim \text{Uniform}(\text{alphabet} \setminus w)$
4. the remaining  $n^2 - n$  letters are placed in the remaining  $n^2 - n$  cells via a uniformly random permutation

with *no rejection step*.  $M_1$  is deliberately the simplest well-posed model, and is known to be wrong in at least one respect: the 2026-09-17 discussion raised the possibility that a real puzzle setter rejects (and retries) any grid whose cross letters accidentally spell a second valid word along some other line. That complication is deferred to a future  $M_2$ ; every result below is explicitly a result *about*  $M_1$ , not a claim about the real puzzle.

The excluded letter  $e$  is told to the player before play starts (as in the real game), so it is fixed per game instance rather than left as a live hypothesis.

## 2.1 Oracle

Querying an untested letter  $L \neq e$  deterministically returns:

- the cell  $p[w.\text{index}(L)]$ , if  $L$  is part of the answer word,
- nothing (no location disclosed), if  $L$  is not part of the answer word.

This matches `wordhunt.py`'s bookkeeping: a “not in the word” answer never discloses where that cross letter actually sits.

## 3 Objective $L_1$

Minimise  $\mathbb{E}[\text{number of queries until } (w, p) \text{ is uniquely determined}]$ .

## 4 Exact DP

Let  $H$  be the current set of *candidate*  $(w, p)$  pairs consistent with everything observed so far. Under  $M_1$ , every candidate world is equally likely (Section 5 makes this precise), so:

$$V(H) = 0 \quad \text{if all remaining candidates share one } (w, p) \quad (1)$$

$$V(H) = 1 + \min_L \sum_{\text{outcome } o} \frac{|H_o|}{|H|} V(H_o) \quad \text{otherwise} \quad (2)$$

where  $H_o$  partitions  $H$  by the outcome querying  $L$  would deterministically produce for each candidate: one bucket for “ $L$  absent”, merged across every candidate not containing  $L$ , and one bucket

per concrete grid cell  $c$  for “ $L$  present at  $c$ ”. This is exactly the outcome-partition derived (for a different, contested weighting) in the wordhunt2.py design discussion; grounding it in an explicit, validated generative model removes that earlier ambiguity.

The  $\arg \min_L$  at the root is the provably optimal next query — not a proxy for it. This is implemented as `ExactSolver` (operating on fully-enumerated worlds) and `PairSolver` (operating directly on  $(w, p)$  pairs) in `solver.py`.

Also implemented: `greedy_choice`, the one-step maximum-information-gain heuristic (i.e. choosing  $L$  to maximise the Shannon entropy of the outcome partition on its own, ignoring everything beyond one query) — the same style of proxy used throughout `wordhunt2.py`. Comparing it against `ExactSolver` is the central point of this exercise.

## 5 The (word, line) sufficiency lemma

Full-world enumeration (build every literal grid  $M_1$  can produce) is combinatorially impossible beyond very small  $n$ : the number of ways to permute the  $n^2 - n$  dross letters into the remaining cells is  $(n^2 - n - 1)!$ .

| $n$ | alphabet size | lines | dross permutations $(n^2 - n - 1)!$ |
|-----|---------------|-------|-------------------------------------|
| 2   | 5             | 6     | $2! = 2$                            |
| 3   | 10            | 8     | $6! = 720$                          |
| 4   | 17            | 10    | $12! = 479,001,600$                 |
| 5   | 26            | 12    | $20! \approx 2.43 \times 10^{18}$   |

**Lemma.** Under  $M_1$ , with the excluded letter  $e$  fixed, every  $(w, p)$  hypothesis has *exactly* the same number of full-world realisations (the dross-permutation count depends only on  $n$ , not on which word or line), so this factor is a constant that cancels out of every relative-probability calculation. The exact DP can therefore be run directly on  $(w, p)$  pairs, weighting each equally, without ever constructing a full grid.

**Verification.** Rather than trust the algebra alone, `tests/test_pair_solver.py` builds both `ExactSolver` (on fully-enumerated worlds) and `PairSolver` (on pairs only) for every excluded-letter scenario at  $n = 2$  and  $n = 3$  — the largest sizes where full enumeration is still tractable — and asserts the two report bit-identical  $V(H_0)$ . Both tests pass (Section 6). Only after this check succeeded was `PairSolver` trusted for  $n = 4$  and  $n = 5$ , where full enumeration is impossible (row  $n = 4$  in the table above is already 479 million permutations *per* (word, line, excluded) choice).

This removes the dross-permutation factorial as an obstacle entirely. It does *not* remove the DP’s own combinatorial cost, which turns out to be the real wall (Section 9).

## 6 Results at $n = 2$

Toy vocabulary (alphabet  $\{a, b, c, d, e\}$ ):  $W = \{ab, bc, cd, de, ea\}$  (deliberately cyclic/symmetric).

| excluded | candidate words | optimal $\mathbb{E}[\text{queries}]$ | greedy $\mathbb{E}[\text{queries}]$ |
|----------|-----------------|--------------------------------------|-------------------------------------|
| a        | 3               | 2.2778                               | 2.2778                              |
| b        | 3               | 2.2778                               | 2.2778                              |
| c        | 3               | 2.2778                               | 2.2778                              |
| d        | 3               | 2.2778                               | 2.2778                              |
| e        | 3               | 2.2778                               | 2.2778                              |

**Finding 1.** At  $n = 2$ , greedy exactly matches the true optimum in every scenario tested. This looks like a validation of the greedy heuristic, but Section 7 shows it is an artefact of  $n = 2$  being too small/symmetric a case for greedy’s known failure mode to appear — not evidence that greedy is generally optimal.

Every  $\mathbb{E}[\text{queries}]$  figure above was independently cross-checked by `simulate_policy_exact_expectation`, an independently-coded exact recursion that replays the solver’s chosen policy against every one of the 36 enumerated worlds per scenario and recomputes the same expectation from scratch (`test_exact_solver_value_matches_independent_simulation`).

## 7 Results at $n = 3$ : greedy is provably suboptimal

Toy vocabulary (alphabet  $\{a..j\}$ ):  $W = \{\text{abc, cde, efg, ghi, ijb, bad, dec, fah}\}$ .

| excluded | candidate words | worlds | optimal       | greedy        |
|----------|-----------------|--------|---------------|---------------|
| a        | 5               | 28800  | 2.3000        | 2.3000        |
| b        | 5               | 28800  | 2.3250        | 2.3250        |
| <b>c</b> | 5               | 28800  | <b>2.5000</b> | <b>2.5500</b> |
| <b>d</b> | 5               | 28800  | <b>2.5250</b> | <b>2.5500</b> |
| e        | 5               | 28800  | 2.3250        | 2.3250        |
| <b>f</b> | 6               | 34560  | <b>2.5000</b> | <b>2.5417</b> |
| <b>g</b> | 6               | 34560  | <b>2.5000</b> | <b>2.5417</b> |
| h        | 6               | 34560  | 2.5000        | 2.5000        |
| <b>i</b> | 6               | 34560  | <b>2.5208</b> | <b>2.5417</b> |
| <b>j</b> | 7               | 40320  | <b>2.6964</b> | <b>2.7321</b> |

**Finding 2 (the headline result so far).** In 6 of the 10 scenarios, one-step maximum-entropy greedy selection is *strictly* worse than the true optimum — not by a rounding error, but by a consistently reproducible margin (e.g. excluded=‘c’: 2.55 vs. 2.50). This is an exact, exhaustively-verified counterexample to trusting greedy one-step information gain as “the” optimal-play criterion, of exactly the kind known from optimal Mastermind/Wordle solvers: two letters can tie (or nearly tie) on immediate information gain while differing in how good a position they leave for the *next* query. Every number above is exact (not simulated) and was solved in under 1.5s per scenario (`research/tests`, and the ad-hoc script in the git history of this branch).

This directly undercuts every “pick the highest-entropy/highest-Bayes-mass letter” heuristic used by `wordhunt2.py` to date: even granting their weighting assumptions, one-step greedy selection on top of them is demonstrably not the same thing as optimal play.

## 8 The $n = 4$ wall (diagnosed, and defused)

Full-world enumeration is impossible at  $n = 4$  (Section 5’s table:  $12! \approx 4.8 \times 10^8$  dross permutations per (word, line, excluded) choice). Thanks to the sufficiency lemma, this does not matter: `PairSolver` needs no full worlds, and its state space at  $n = 4$  is just (number of candidate words)  $\times$  (10 lines) – no factorial term at all. The real question is whether *that* state space is tractable, which turns out to depend overwhelmingly on  $n$  (alphabet/branching factor) and vocabulary size rather than on the factorial term that looked scary at first glance.

## 9 The real wall: $n = 5$ with realistic vocabulary size

Using `PairSolver` (validated, not full-world enumeration) directly on random samples of real English words from `dict_game.csv` (excluded letter fixed at ‘h’, matching a real play session from 2026-09-17), starting from the fully-uninformed state:

| sample size (words) | $(w, p)$ pairs | optimal $\mathbb{E}[\text{queries}]$ | wall-clock        | DP states memoised |
|---------------------|----------------|--------------------------------------|-------------------|--------------------|
| 10                  | 120            | 2.583                                | 0.08 s            | 896                |
| 15                  | 180            | 2.894                                | 0.54 s            | 3,441              |
| 20                  | 240            | 3.108                                | 2.57 s            | 11,525             |
| 25                  | 300            | 3.337                                | 5.66 s            | 23,925             |
| 30                  | 360            | 3.381                                | 10.04 s           | 39,725             |
| 40                  | 480            | —                                    | >25 s, incomplete | >58,237            |

Memoised-state growth is well above linear in the number of pairs (roughly consistent with  $O(|H_0|^{3.5})$  across this range, though the sample is too small to fit a reliable exponent): doubling the pair count from 180 to 360 grows the state count by  $\sim 11.5\times$ , not  $2\times$ .

**Finding 3.** The real `dict_game.csv` has 10,176 candidate words, i.e. roughly  $300\times$  the 30-word sample that already took 10 seconds and 40,000 memoised states. Extrapolating even the optimistic end of the observed growth curve, exact full-lookahead solving at real  $n = 5$  scale, from the fully-uninformed opening state, is not a performance-tuning problem — it is many orders of magnitude beyond reach on this hardware, and almost certainly on any hardware, without a fundamentally different algorithmic approach (see Section 10).

Importantly, this wall is a property of  $n$  (i.e. the branching factor: up to 21 queryable letters at  $n = 5$  vs. up to 7 at  $n = 3$ ) and of how *early* in the game the search starts (the fully-uninformed opening state has the largest candidate set the game ever sees), much more than it is a property of vocabulary size on its own: Section 7’s  $n = 3$  instances, with comparable vocabulary sizes (5–8 words) but a 6–7-letter query alphabet instead of a 21-letter one, solved in well under a second.

## 10 Next steps

Not yet attempted, in rough order of expected value:

1. **Branch-and-bound with a provable lower bound.** A node can be pruned the moment its best-case remaining cost (e.g. log of the branching factor) already exceeds the best solution found so far. This preserves exactness (no silent fallback to a heuristic) while potentially cutting large amounts of dead search.
2. **Solve from mid-game states, not the opening.** A real player only needs an exact answer from *wherever they currently are* – typically well past the opening, with the candidate set already substantially pruned by revealed cells. Section 9’s wall is specifically about the worst case (opening, full 10k-word dictionary); it may still be exactly solvable a few queries in, once  $|H|$  has naturally shrunk. Worth measuring directly rather than assuming.
3. **Depth-limited exact search with a reported bound.** Search exactly to some fixed depth  $d$ , and for states beyond that, report a provable interval (not a guess) rather than a heuristic point estimate – matches the “never bluff a heuristic under the optimal-solver label” principle from the design discussion.

4. **Model  $M_2$** : reintroduce the “dross letters must not accidentally spell a second word” rejection step raised on 2026-09-17, and check whether  $M_1$ ’s uniform-pair-weighting result changes qualitatively once that’s modelled, at whatever board size remains tractable.
5. **A different objective  $L_2$** :  $\mathbb{E}[(\text{queries})^2]$  requires tracking two moments through the Bellman recursion instead of one; algebraically routine but not yet implemented.

It is entirely possible that the honest endpoint of this project is a short paper whose conclusion is: exact optimal play is achievable and verified at toy scale ( $n \leq 3$ ), is real and worth verifying at  $n = 5$  from *sufficiently-advanced* mid-game states, but is fundamentally out of reach for the opening move against a realistic dictionary – and greedy one-step heuristics, however dressed up, are not a substitute for it (only a proxy of unknown and state-dependent quality, per Section 7). That would be a complete and legitimate result, not a failure to reach one.

## 11 Corrections (2026-09-17, same day)

Per this logbook’s stated practice, errors found in the results above are recorded here rather than silently rewritten. **Every numeric result in Sections 6–9 was computed under a wrong oracle model** and should be treated as superseded pending re-derivation.

### 11.1 Bug 1: the oracle model was wrong

The original `query_outcome` returned *no location at all* when a queried letter was not part of the answer word. This is not what the real game does, and not what `wordhunt.py` implements: the grid is a full bijection over every non-excluded letter, so querying any letter always resolves to its one true cell, whether or not that letter turns out to be part of the answer. A “not in the word” outcome never occurs as a distinct case in `wordhunt.py`’s bookkeeping – every entry in `revealed` is handled identically by `word_fits_position` regardless of whether it later turns out to be dross. `query_outcome` has been corrected to always return the letter’s true cell.

This is the same category of mistake as the earliest abandoned “F(L|G)” attempt from the original design discussion (modelling where a dross letter “would be”), recurring in a new piece of code – worth remembering next time an oracle-behaviour assumption is written down without a test against brute-force enumeration first.

### 11.2 Bug 2: the stopping condition was ill-posed without a veto step

Fixing bug 1 immediately exposed an infinite loop in the DP. Diagnosis: at  $n = 2$ , `excluded=‘e’`, two *different* generative paths –  $w = \text{“ab”}$  along the top row, and  $w = \text{“cd”}$  along the bottom row – can produce the *identical* grid:

```
World A: word='ab', line=top row grid={(0,0):a,(0,1):b,(1,0):c,(1,1):d}
World B: word='cd', line=bottom row grid={(0,0):a,(0,1):b,(1,0):c,(1,1):d}
SAME GRID? True
```

No sequence of oracle queries can ever tell these apart, because the oracle’s answers are a pure function of the grid, and the grid is identical. The DP’s termination check (“is the tagged (word,line) unique?”) was asking an unanswerable question and looped forever trying to resolve it. This is exactly the concern raised when  $M_1$ ’s lack of a veto step was agreed to: “if you choose to do that you must then be clear about the stopping condition.” It surfaced as a literal bug rather than staying a hypothetical.

**Fix:** termination is now defined on *grid* uniqueness, not tagged-pair uniqueness – i.e.  $M_1$ ’s well-defined notion of ”solved” is ”the grid is fully reconstructed,” which coincides with ”the designated answer is known” except exactly in a collision, where the latter is genuinely unknowable and the former is the most that can honestly be claimed.

**Measured collision rate** (fraction of raw, unvetted  $M_1$  worlds whose grid happens to spell more than one dictionary word somewhere):

| $n$ | vocabulary size | collision rate                          |
|-----|-----------------|-----------------------------------------|
| 2   | 5 words         | 69% (all excluded-letter scenarios)     |
| 3   | 5–7 words       | 4.3%–8.2%, depending on excluded letter |

The  $n = 2$  figure is inflated by an unrealistically small, densely overlapping toy vocabulary (a 5-word cycle on a 5-letter alphabet), but the  $n = 3$  figures are a genuine, measured answer to ”how significant a departure is it to skip the veto step?”: even a tiny 5–7-word vocabulary already produces an ambiguous grid 1 time in 12–25. This is direct evidence (not argument) that  $M_2$  (with the veto step) is not an optional refinement – it is needed for  $M_1$  to be a reasonable model of the real game at all, exactly as anticipated in the original design discussion.

### 11.3 Consequence: exact solving got much harder, not easier

The corrected oracle is more informative per query (up to  $n^2$  distinct outcomes – which cell – versus the old model’s effectively  $\leq n + 1$ ), which sounds like it should make the game easier to solve. It does not make the *search* easier: richer outcomes mean a larger branching factor at every DP node, and the number of distinct reachable candidate sets explodes correspondingly. Concretely, at  $n = 3$ , excluded=’a’ (5 candidate words, only 28,800 worlds – trivial under the old, wrong oracle, solved in  $< 1$ s with  $\sim 100$  memoised states): under the corrected oracle, **ExactSolver** passed 700,000 memoised states without terminating within 20 seconds.  $n = 2$  remains fully tractable (re-solved below).

### 11.4 Consequence: the (word, line) sufficiency lemma is retracted

Section 5’s lemma held under the old oracle because a dross letter’s outcome (”absent”) genuinely didn’t depend on anything beyond (word, line). Under the corrected oracle, a dross letter’s revealed cell depends on the specific permutation of the remaining letters, which is *not* determined by (word, line) alone, and whose distribution over remaining cells stops being uniform across surviving pairs once some cells have already been revealed (precisely the complication that broke the very first F(L|G) attempt). **PairSolver** still silently implements the old model and is simply wrong now; its equivalence tests are kept in `tests/test_pair_solver.py` but marked `xfail/skip` with an explanation, as the documented record of the retraction rather than being deleted. A correct pair-based (i.e. not full-world) solver requires properly counting consistent completions per pair given the revealed history so far – not yet attempted.

### 11.5 Re-solved: $n = 2$ , corrected oracle and termination

| excluded | worlds | optimal $\mathbb{E}[\text{queries}]$ | greedy $\mathbb{E}[\text{queries}]$ |
|----------|--------|--------------------------------------|-------------------------------------|
| a        | 36     | 2.944                                | 2.944                               |
| b        | 36     | 2.944                                | 2.944                               |
| c        | 36     | 2.944                                | 2.944                               |
| d        | 36     | 2.944                                | 2.944                               |
| e        | 36     | 2.944                                | 2.944                               |

Greedy still matches optimal at  $n = 2$  (as before the fix – still plausibly just an artefact of this toy instance’s size/symmetry; Section 7’s  $n = 3$  counterexample was not re-verified under the corrected oracle because **ExactSolver** does not currently finish at  $n = 3$  post-fix – see above). Note the value itself changed from the (wrong) 2.278 to 2.944: full-grid termination is a strictly harder target than tagged-pair termination, so this increase is expected, not a sign of a new bug, *conditional on “know the whole grid” being the right definition of winning* – which is exactly the open question below.

### 11.6 Open question, not yet resolved: what does “winning” require?

Grid-uniqueness termination is *sound* (it can’t loop forever, and it never claims certainty it doesn’t have), but it may be stricter than necessary. The real game’s actual win condition is presumably “the player can read off some line that is certainly a complete, valid word” – not “every dross cell is known.” Those two conditions coincide throughout almost the entire game, and differ only in collision states, where “the designated word” is unknowable but “some certainly-correct line” might still not require full grid knowledge either way. A closer-to-real definition would be: stop as soon as some  $(w, p)$  is spelled by *every* remaining consistent grid (not necessarily the one tagged as “designated,” and not necessarily requiring every other cell to be pinned down too). This was noticed, not yet implemented, and the 2.944 figure above should be read as an upper bound on expected queries under the real win condition, not necessarily the answer to the question we actually care about.